

MeshConv

The four differential operators of the spherical U-Net

Jiang et al. 2019 — a study guide

The MeshConv equation

Everything in this guide serves one formula. A MeshConv layer replaces the fixed 3×3 square kernel of an ordinary CNN with a small basis of **geometric measurements** taken on the mesh:

$$F * G_{\theta}^{\text{diff}} = \theta_0 IF + \theta_1 \nabla_{\text{EW}} F + \theta_2 \nabla_{\text{NS}} F + \theta_3 \nabla^2 F$$

Read left to right, the symbols mean:

F one feature defined on the mesh vertices (one number per vertex).

IF the **original** feature value, untouched.

$\nabla_{\text{EW}} F$ the rate of change of F toward the east–west direction.

$\nabla_{\text{NS}} F$ the rate of change of F toward the north–south direction.

$\nabla^2 F$ how the feature **bends / deviates** relative to its surrounding neighbourhood.

$\theta_0, \theta_1, \theta_2, \theta_3$ four **learned** coefficients.

The geometry operators are fixed from the mesh.

The four θ values are learned.

geometry decides **how** to measure the local pattern



the network learns **how much** each measurement matters

The rest of this guide unpacks each of the four operators, from physical meaning down to the exact number produced at a single vertex, and finally how MeshConv consumes those numbers.

Before the operators: what is F ?

F is not an image and not a vector field. It is simply **one feature value stored at every vertex** of the mesh. If the feature is cortical curvature:

```
vertex 0    -> curvature =  0.12
vertex 1    -> curvature =  0.18
vertex 2    -> curvature = -0.05
...
vertex 10241 -> curvature =  0.09
```

Collected into a column, that is the whole of F :

$$F = \begin{pmatrix} F_0 \\ F_1 \\ F_2 \\ \vdots \\ F_{10241} \end{pmatrix} \quad \text{shape } [V]$$

where V is the number of vertices (for the paper's level-5 icosphere, $V = 10\{, \}242$). With a batch of B examples and C feature channels the tensor is:

$$[B, C, V]$$

Key idea. Every differential operator below receives these vertex values and returns **one new value at every vertex**. Input $[V]$, output $[V]$. The operators differ only in **what they measure** about the local neighbourhood.

Operator 1 — Identity I

Intuitive meaning

Identity asks only one question: “**what is the feature value at this vertex?**” It adds no neighbourhood information whatsoever. If

vertex X : curvature = 0.8

then

$$I(F)(X) = 0.8$$

The formula

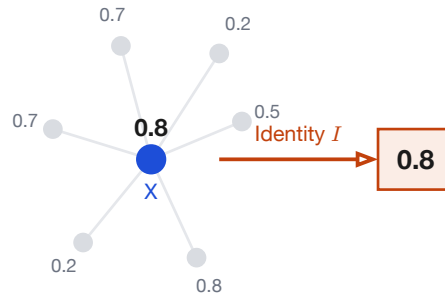
$$IF = F \quad \text{or at vertex } i: (IF)_i = F_i$$

No approximation is required — this is the zeroth-order operator.

Matrix picture

$$I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad IF = F$$

Row i reads **only** vertex i ; nothing enters from the neighbours. For cortical data at vertex 4200 with curvature 0.8, the Identity branch simply preserves 0.8. This lets MeshConv learn from the **absolute** feature value, not only from how it changes.



Identity: preserve the value at the current vertex — no neighbour arrows are used.

Operator 2 — Gradient

Before east–west and north–south, we need the ordinary derivative.

The ordinary derivative (1-D)

position:	0	----	1	----	2
value:	2		5		9

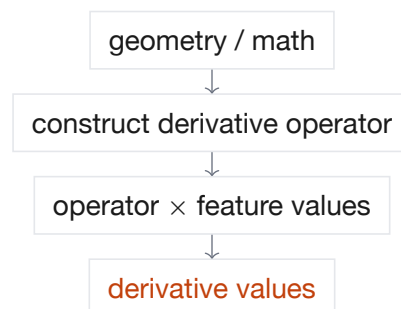
A derivative asks: **how quickly is the signal changing as I move?** The central difference estimates it from the two neighbours:

$$\frac{dF}{dx} \approx \frac{F_{\text{right}} - F_{\text{left}}}{2h} \Rightarrow \frac{9 - 2}{2} = 3.5 \quad (h = 1)$$

The same number is a **dot product** of a fixed row with the values:

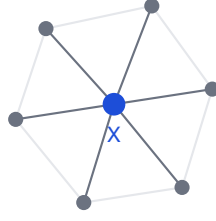
$$(-0.5 \ 0 \ 0.5) \begin{pmatrix} 2 \\ 5 \\ 9 \end{pmatrix} = 3.5$$

Matrix multiplication does not magically create the derivative. The derivative formula is already **encoded in the coefficients** $(-0.5 \ 0 \ 0.5)$. Multiplying by the feature values just evaluates that formula.



Why gradients are harder on a mesh

On a raster image the four directions **left / right / up / down** are obvious because pixels sit on a rectangular grid, so a derivative can be literally “right pixel – left pixel”. On a triangular mesh the neighbours are **not** on a square:



So the derivative cannot be a fixed “right – left”. Instead the mesh **geometry** — vertex coordinates, triangles, triangle areas, and tangent directions — supplies the coefficients.

How the paper computes the mesh gradient

A continuous scalar feature is approximated as a **piecewise-linear** function over the triangles, built from “hat” basis functions φ_i :

$$f(x) = \sum_{i=1}^n \varphi_i(x) f_i$$

where f_i is the value at vertex i , and $\varphi_i = 1$ at vertex i and 0 at every other vertex. Differentiating,

$$\nabla f(x) = \sum_{i=1}^n \nabla \varphi_i(x) f_i.$$

Because each triangle interpolates **linearly**, its gradient is **constant inside that triangle**. The paper writes this per-face gradient as

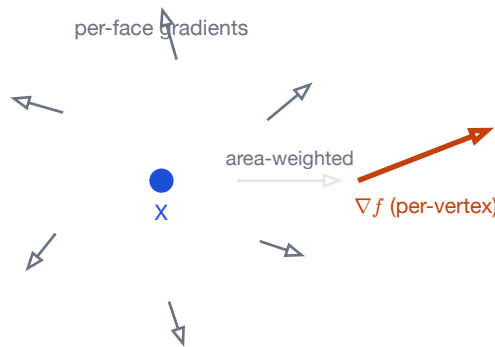
$$\nabla f^{\text{face}} = G f^{\text{vertex}}$$

where G is a fixed operator built from the triangle’s coordinates and its finite-element basis (we do not derive G here — it comes from the geometry).

A single vertex touches **several** triangles, each with its own gradient vector. The paper averages them, weighting each face by its area:

$$\nabla f_i^{\text{vertex}} = \frac{\sum_{j \in N(i)} a_j^{\text{face}} \nabla f_j^{\text{face}}}{\sum_{j \in N(i)} a_j^{\text{face}}}.$$

Larger triangles contribute proportionally more.



Operator 2a — East–West gradient

The full gradient ∇f is a **vector** along the surface — direction and speed of fastest increase. MeshConv wants **directional** information, so the paper defines a local unit tangent $\hat{e}_{\text{EW}}$ pointing east–west and takes the component of the gradient along it:

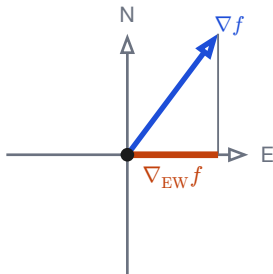
$$\nabla_{\text{EW}} f = \nabla f \cdot \hat{e}_{\text{EW}}.$$

This dot product asks: **how much of the full gradient points east–west?**

- gradient points strongly east $\Rightarrow$ large **positive** EW
- strongly west $\Rightarrow$ large **negative** EW
- gradient almost north–south $\Rightarrow$ EW ≈ 0

Numerical example. If $\nabla f = (3, 4)$ and $\hat{e}_{EW} = (1, 0)$ then

$$\nabla_{EW} f = (3, 4) \cdot (1, 0) = 3.$$



The EW derivative is the **component of the surface gradient pointing east–west**: the orthogonal projection of ∇f onto the east–west tangent axis.

Operator 2b — North–South gradient

Exactly the same logic with the perpendicular unit tangent $\hat{e}_{NS}$:

$$\nabla_{NS} f = \nabla f \cdot \hat{e}_{NS}.$$

With $\nabla f = (3, 4)$ and $\hat{e}_{NS} = (0, 1)$:

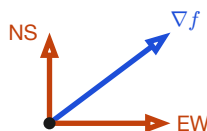
$$\nabla_{NS} f = (3, 4) \cdot (0, 1) = 4.$$

Sign: **positive** $\Rightarrow$ increases toward the chosen north; **negative** $\Rightarrow$ increases the opposite way; **near zero** $\Rightarrow$ little change along that axis.

The paper constructs east–west and north–south **direction fields** over the sphere and projects the per-vertex gradient onto those unit tangents. So EW and NS are **not** arbitrary labels attached to neighbours — they are genuine **directional components of the tangent-gradient approximation**.

Full gradient vs. its two components

∇f is a single tangent vector. MeshConv does **not** feed the whole vector in as one quantity — it separates it into two **scalar** directional components, $\nabla_{EW} f$ and $\nabla_{NS} f$, giving the kernel directional information.



Operator 3 — Laplacian ∇^2

This operator deserves more than “neighbours minus centre.”

Continuous meaning

In a flat 2-D domain,

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

The **first** derivative asks “is the value changing?”; the **second** derivative asks “is the **rate of change itself** changing?” So the Laplacian measures the local **bending / concavity** of the scalar signal.

Crucial: this is the Laplacian of the **feature** f living on the surface — it is **not** the geometric curvature of the cortical surface itself. If the input feature happens to be cortical curvature, then $\nabla^2 f$ measures **how that curvature value varies** across the mesh, not the shape of the cortex.

1-D second difference (intuition)

The second difference $f_{\text{left}} - 2f_{\text{center}} + f_{\text{right}}$ captures peakiness. Sharp peak $[1, 5, 1]$: $1 - 2(5) + 1 = -8$. Flat $[5, 5, 5]$: $5 - 2(5) + 5 = 0$. Large magnitude $\Rightarrow$ the centre is a strong peak/valley relative to its neighbours; zero $\Rightarrow$ locally flat.



The Laplacian on a triangular mesh

We cannot use “left $-$ 2 center $+$ right” because the neighbours are not equally spaced, there are several of them, the triangles have different angles, and the surface is curved. The paper uses the **cotangent discretisation** of the mesh Laplace–Beltrami operator:

$$\nabla^2 F_i \approx \frac{1}{2A_i} \sum_{j \in N(i)} (\cot \alpha_{ij} + \cot \beta_{ij})(F_i - F_j)$$

Sign convention. Some texts write $(F_j - F_i)$ instead. That is only a sign convention on the Laplacian; follow the sign used in the Jiang paper when reproducing its formulation.

Symbol by symbol:

i the current centre vertex.

$j \in N(i)$ each one-ring neighbour vertex.

F_i, F_j feature values at centre and neighbour.

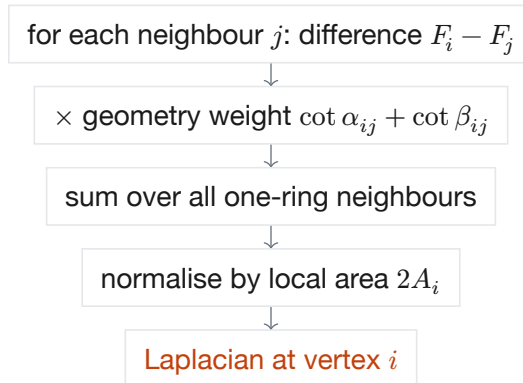
$F_i - F_j$ how different the centre is from that neighbour.

α_{ij}, β_{ij} the two angles **opposite the edge** ij (one in each adjacent triangle).

$\cot \alpha + \cot \beta$ the geometry-derived **weight** of that edge.

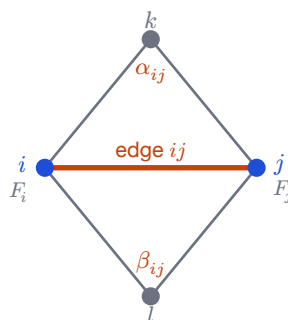
A_i the dual area associated with vertex i .

$\sum$ combine contributions from all one-ring neighbours.



The two angles α_{ij} and β_{ij}

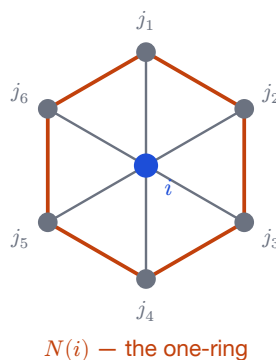
The edge ij is shared by two triangles. In the upper triangle the third vertex is k ; in the lower it is l . The weight uses the angles **at k and at l** — the angles **opposite** the edge ij — **not** the angles at i or j .



$$w_{ij} = \cot \alpha_{ij} + \cot \beta_{ij} \quad \text{neighbour contribution} = w_{ij}(F_i - F_j)$$

The one-ring neighbourhood $N(i)$

$N(i)$ is simply the set of vertices **directly connected** to i . The differential operators need **only** this local ring — which is exactly why MeshConv stays local, like a 3×3 CNN kernel.



Approximate vs. exact

Continuous world. If f were an analytic function on a perfectly continuous sphere, ∇f and $\nabla^2 f$ would be **exact** continuous derivatives.

Mesh world. We only know values $F_0, F_1, F_2, \dots$ at **discrete** vertices, so the exact continuous derivatives are **unavailable**. We **estimate** them from triangle geometry + neighbour values + discrete-differential-geometry formulas.

The network does not learn how to calculate derivatives.
The mesh geometry determines the derivative approximation.
The network learns how strongly to use each derivative.

From operators to tensors

Each operator is a linear map $\mathbb{R}^V \rightarrow \mathbb{R}^V$. Conceptually it is a matrix:

$$[V, V] \times [V] \rightarrow [V]$$

For one feature, $L @ F$ yields one Laplacian value at every vertex; likewise $EW @ F$, $NS @ F$, $I @ F$ each return shape $[V]$. Stacking the four:

$$I @ F, L @ F, EW @ F, NS @ F \rightarrow \text{stack} \rightarrow [V, 4]$$

and with channels and batches, $[B, C, V, 4]$.

Implementation-accuracy note. An educational version may store L , EW , NS as dense $[V, V]$ matrices because that is easy to read. Jiang's **actual** code does **not**: it computes

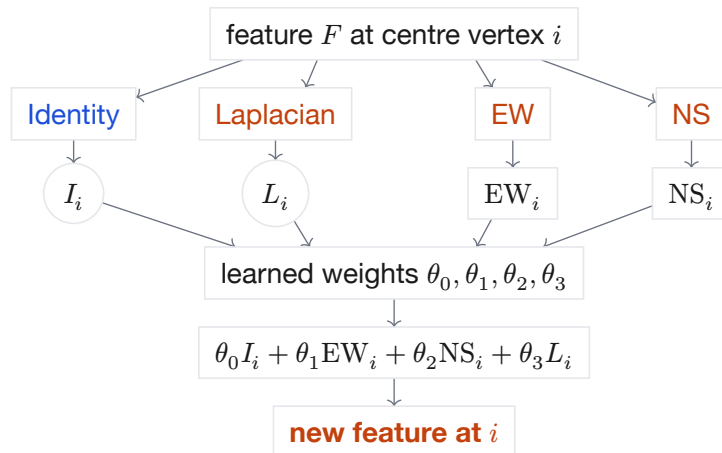
vertex features $\rightarrow$ per-face gradient $\rightarrow$ area-weighted face- $\rightarrow$ vertex average $\rightarrow$ project onto EW/NS tangents

using **sparse** matrices for efficiency. So

$$\text{east_west} = x @ EW.T$$

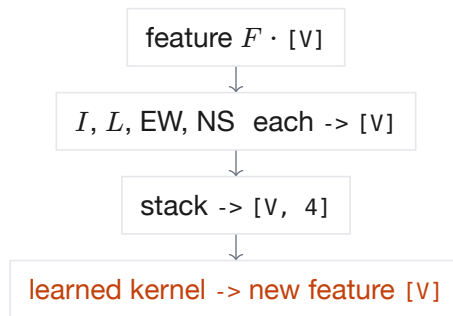
is the **conceptual** final linear operator — do not read the dense $[V, V]$ form as the literal production implementation.

Complete flow at ONE vertex



Term order matches the MeshConv equation: $\theta_0 I + \theta_1 \nabla_{EW} + \theta_2 \nabla_{NS} + \theta_3 \nabla^2$.

Complete flow at ALL vertices



Generalised to batches and multiple channels, the layer maps

$$[B, C_{in}, V, 4] + \text{kernel } [C_{out}, C_{in}, 4] \rightarrow [B, C_{out}, V]$$

MeshConv vs. a 3×3 convolution

Standard 3×3 convolution

A learned kernel of spatial coefficients — the grid gives consistent positions:

$$\begin{pmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{pmatrix}$$

MeshConv

No universal square neighbourhood exists, so a small **geometric basis** is fixed and only its combination is learned:

$$\text{Identity} \cdot \text{EW} \cdot \text{NS} \cdot \text{Laplacian} \rightarrow \text{learn } \theta_0, \theta_1, \theta_2, \theta_3$$

A standard CNN learns weights for fixed grid positions.

MeshConv learns weights for fixed geometric differential measurements.

Summary

IDENTITY

“What is the value here?”

EW GRADIENT

“How fast does the feature change east–west?”

NS GRADIENT

“How fast does the feature change north–south?”

LAPLACIAN

“How is the feature locally bending / deviating relative to its one-ring neighbourhood?”

geometry computes these four measurements

MeshConv learns how to combine them

new feature at every vertex

Final warning. Do not confuse the Laplacian of a cortical **feature** with the physical curvature of the cortical **mesh**. If the input feature itself is curvature, the Laplacian measures how that curvature **value** changes across the surface — not the shape of the cortex.

Appendix — Prompts for generating operator visualizations

Identity. “Scientific vector diagram of a triangular spherical mesh patch. Highlight one center vertex X with scalar value 0.8. Neighbor vertices are visible but faded. Show an arrow from X directly to an output value 0.8 labeled Identity I . No neighbor contributions. White background, clean educational style.”

Gradient construction. “Scientific infographic of a triangular mesh around center vertex X . Each incident triangular face contains a small gradient vector. Show face areas and weighted arrows converging toward X , producing one area-weighted per-vertex gradient vector ∇f . Clearly distinguish per-face gradients from the final vertex gradient.”

East–West derivative. “Scientific tangent-plane diagram at one vertex on a sphere. Show local east-west and north-south tangent axes and a diagonal full surface gradient vector ∇f . Orthogonally project ∇f onto the east-west axis and highlight that component as $\nabla_{EW} f = \nabla f \cdot e_{EW}$.”

North–South derivative. “Scientific tangent-plane diagram at one vertex on a sphere. Show local east-west and north-south tangent axes and a diagonal full surface gradient vector ∇f . Orthogonally project ∇f onto the north-south axis and highlight that component as $\nabla_{NS} f = \nabla f \cdot e_{NS}$.”

Laplacian. “Scientific triangular-mesh diagram explaining the cotangent Laplacian. Highlight center vertex i and neighbor j sharing an edge. Show the two triangles adjacent to edge ij and clearly mark the two angles α_{ij} and β_{ij} opposite that edge. Show F_i and F_j beside the vertices. Beside the mesh show contribution $(\cot \alpha_{ij} + \cot \beta_{ij})(F_i - F_j)$. Extend faintly to other one-ring neighbors to indicate that all contributions are summed.”

Full MeshConv. “Scientific flowchart showing one scalar feature defined on a spherical triangular mesh entering four parallel branches: Identity I , East-West derivative, North-South derivative, and Laplacian. Each branch produces one scalar per vertex. The four outputs enter learned coefficients $\theta_0, \theta_1, \theta_2, \theta_3$, are summed, bias is added, and a new feature map over all mesh vertices is produced. Include tensor annotations $[V] \rightarrow \text{four } [V] \text{ signals} \rightarrow [V, 4] \rightarrow \text{learned combination} \rightarrow [V]$.”